

dyablo FoF: user guide

A parallel Friends-of-Friends halo finder for dyablo HDF5 particle snapshots. Adapted from the RAMSES-era pFoF (Roy/Audit) tool. This guide covers building, running, and post-processing the output to produce a halo mass function plot against theory.

1 What you get

The shipped directory contains:

- `*.f90`, `makefile` — the Fortran sources and build system.
- `fof.in` — example input file (one line per parameter).
- `plot_hmf.py` — Python script that reads the FoF catalog and overplots Sheth–Tormen / Tinker08 at the snapshot redshift.

2 Build

Requirements: an MPI-aware Fortran compiler (`mpif90`) and HDF5 with Fortran bindings.

The `makefile` is set up for macOS with Homebrew (`brew install hdf5 open-mpi`). The only two lines you typically have to touch are at the top:

```
HDF5_DIR ?= /opt/homebrew # <-- path to HDF5 install prefix
                        # (must contain include/ and lib/)

OPTS ?= -O3 -cpp -DLONGINT \ # -DLONGINT activates 8-byte particle
        -I$(HDF5_DIR)/include \ # IDs; required for N_p > 2^31.
        -ffree-line-length-none # GFortran-specific; remove for ifort.

LIBS ?= -L$(HDF5_DIR)/lib -lhdf5_fortran -lhdf5
```

Override on the command line if needed:

```
make HDF5_DIR=/usr/local # custom HDF5 prefix
make MPIFC=mpiifort FC=mpiifort # Intel MPI / ifort
```

Then simply:

```
make
```

produces the `fof` executable. `make clean` removes objects and the binary.

3 The `fof.in` input file

`fof.in` is read line by line — order matters; inline comments are allowed only on numerical/logical lines (lines read with list-directed input), not on string lines.

```
my_halos ! root: output filename prefix
DY ! code_index (must be 'DY' for dyablo)
/path/to/snapshot/dir ! pathinput: dir containing the .h5 and .ini
star_particles_..._iter0070000.h5 ! particle HDF5 file
restart_star_NNNNNN.ini ! the dyablo restart .ini
0 !grpsize (RAMSES legacy, leave 0)
```

```

0.2 !percolation parameter b (in mean-inter-particle units)
100 !min mass (in particles): halos with fewer particles are not written
100000000 !max mass (in particles)
.true. !star: snapshot contains star particles to filter out
.false. !metal: never .true. for dyablo
.false. !outcube: write per-rank particle cubes after read (debug)
.true. !dof: actually run halo detection (set .false. to just read+exit)
.false. !readfromcube: read particles from precomputed cubes (debug)
.true. !dotimings: extra MPI syncs to print accurate timings
.false. !invent_ids: force-invent IDs even when snapshot has /id (legacy)
outputs !outdir: where to write outputs (created if missing)

```

Notes:

- `root`, `pathinput`, `namepart`, `nameinfo`, and `outdir` are read as 80-char strings; do not append comments on those lines.
- DM particle mass is *auto-detected* as the median of the `mass` dataset, so you don't have to set it.
- By default the code uses the original `/id` dataset if present. Set `invent_ids = .true.` only if your snapshot has duplicate or corrupt IDs.

4 Run

```

mpirun -np 1 ./fof # serial reference
mpirun -np 8 ./fof # 2 x 2 x 2 spatial decomposition
mpirun -np 64 ./fof # 4 x 4 x 4

```

Rank-count constraint. The MPI decomposition is a cubic $n_{\text{sd}} \times n_{\text{sd}} \times n_{\text{sd}}$ grid with $n_{\text{sd}} = \lfloor N_{\text{proc}}^{1/3} \rfloor$, and the particle grid n_{res} (from `particle_grid.nx` in the `.ini`) must be divisible by n_{sd} . For $n_{\text{res}} = 256$ the valid rank counts are therefore 1, 8, 64, 512, ... (i.e. cubes of powers of two).

5 Output files

In `outdir/`:

- `root.log` — run log (parameters, timings, halo count).
- `root.inp` — echo of the input parameters.
- `root_masst_NNNNN` — one file per MPI rank: halo catalog (Fortran sequential unformatted). Each halo record is `int64 id`, `int32 mass`, `3*float32 (x,y,z)` with positions normalized to $[0, 1]^3$.
- `root_strct_NNNNN` — per-halo particle membership (positions, velocities, and original snapshot IDs).

6 Plotting the halo mass function

`plot_hmf.py` reads the `*_masst_*` files, pulls the cosmology from the dyablo `.ini` and the current scale factor from the snapshot `.h5` (`scalar_data/aexp`), and overplots two analytic HMFs (Sheth-Tormen and Tinker08) at the same redshift.

```

python3 plot_hmf.py \
  --catalog outputs/ \
  --ini /path/to/restart_star_NNNNNN.ini \
  --h5 /path/to/star_iterNNNNNNN.h5

```

Optional flags: `-sigma8` (default 0.81), `-ns` (default 0.96), `-hmf-model` (default SMT; any model name accepted by the `hmf` package), `-out` (output PNG path, default `hmf_compare.png`).

Python dependencies: `numpy` `scipy` `h5py` `matplotlib` `hmf` `astropy`.

Note: σ_8 and n_s are not stored in the dyablo `.ini`; the script uses Planck-ish defaults. Override with `-sigma8/-ns` if your ICs use different values.

7 Troubleshooting

- *Build fails on Cannot open .mod:* HDF5 Fortran module not found. Check `$(HDF5_DIR)/include`.
- *Run dies in dyablo_lecture:* usually a path issue — check `pathinput`, `namepart`, `nameinfo`.
- *Crash in MPI_Finalize on a laptop* (macOS, OFI `utun*` errors): harmless — the FoF work is already done and output files are written. Set `FI_PROVIDER=shm` or disconnect the VPN to silence it. Does not occur on HPC clusters.

8 History and acknowledgements

This tool descends from the parallel Friends-of-Friends code originally written for RAMSES snapshots: Edouard Audit’s sequential FoF (CEA), parallelised by Fabrice Roy (CNRS / LUTH, Observatoire de Paris), with further contributions from Yann Razera.

The adaptation to dyablo HDF5 snapshots was developed across two interactive coding sessions with Claude Opus 4.7 (Anthropic):

- *Port.* Replaced the RAMSES particle reader with a dyablo HDF5 reader (`dyablo_lecture`); added a minimal parser for the dyablo restart `.ini` to recover box geometry and the particle grid resolution; auto-detection of the DM particle mass as the median of the `mass` dataset (so the mass no longer has to be hard-coded); optional filtering of star particles via the snapshot’s `birth_time` dataset; preservation of the snapshot’s `/id` dataset by default (the legacy “invent IDs” behaviour is now opt-in via `invent_ids = .true.`); removal of RAMSES-only code paths.
- *Validation and packaging.* Configurable output directory (`outdir`) so runs no longer clutter the source tree; cross-rank reproducibility checks (catalog bit-exact at 1 and 8 ranks; the few-particle drift seen at 64 ranks is consistent with floating-point ambiguity at the larger number of rank boundaries); halo-mass-function comparison against Sheth–Tormen and Tinker08, with cosmology pulled automatically from the dyablo `.ini` and the current scale factor from `scalar_data/aexp`; per-halo particle visualisation script for sanity-checking the cosmic web; cross-platform build verified on macOS / Homebrew and on Linux / Debian (`h5pfc`); this user guide.

The Fortran sources retain the original authorship attributions at the top of each module. The dyablo branch was developed and validated with the assistance of Claude Opus 4.7.